

Práctica de Árboles

Programación II – ALGORITMOS Y ESTRUCTURAS DE DATOS II
- UADE 2026

En los archivos de la materia encontrarán un proyecto base con:

- La interfaz **ABBEnteros**
- Una implementación parcial o incompleta **ABBEnterosImpl**
- Una batería de **tests** de validación

El objetivo es completar la implementación de un Árbol Binario de Búsqueda de enteros.

Consideraciones

- La estructura debe implementarse utilizando nodos enlazados.
- Se deben utilizar recursión en la mayoría de los métodos, salvo que se indique lo contrario.
- No se permite el uso de estructuras auxiliares de Java (List, Set, Map, etc.) para resolver los ejercicios.
- No se debe modificar la interfaz ABBEnteros ni los Tests.
- Los recorridos deben imprimirse por consola.

Ejercicio 1 - Inserción recursiva

Implementar el método void **agregar**(int valor) utilizando **recursión**.

Tener en cuenta que no se permiten duplicados, por lo tanto agregar un valor existente no tiene efecto.

Indicar que complejidad posee en peor caso.

Ejercicio 2 - Inserción iterativa

1. Ejecutar la función main de la clase **TestRecursividadABB**. Luego analizar el código y:
 - a. Describir qué está haciendo el método main.
 - b. Explicar qué problemática se evidencia y cómo se relaciona con las implementaciones recursivas e iterativas?
 - c. ¿Por qué ocurre un StackOverflowError?
2. Implementar el método public void **agregarIterativo**(int valor), que permite agregar elementos (sin repetidos) pero utilizando bucles en vez de recursión.
3. Comparar ambas implementaciones y describir ventajas y desventajas de ambas.

Ejercicio 3 - Búsqueda

Implementar el método boolean **contiene**(int valor).

Indicar Complejidad temporal en el peor caso.

Comparar búsqueda en un ABB con búsqueda lineal y búsqueda binaria en un Array.

Ejercicio 4 - Estadísticas del árbol

Implementar

- int altura();
- int cantidad();
- int minimo();
- int maximo();

* Los métodos mínimo y máximo lanzan un error si se usan en un árbol vacío.

Ejercicio 5 - Recorridos

Implementar los métodos

- mostrarInOrden
- mostrarPreOrden
- mostrarPostOrden

Qué particularidad observan en el resultado de mostrarInOrden?

Indicar complejidad temporal.

Ejercicio 6 - Eliminación

Implementar el método boolean **eliminar**(int valor).

La idea general del código es:

- Si el nodo es null:
 - termino (no hay nada que hacer)
- Si el valor es menor:
 - sigo buscando por la izquierda
- Si el valor es mayor:
 - sigo buscando por la derecha
- Si encuentro el nodo que quiero borrar:
 - caso 1: es hoja
 - lo elimino directamente (devuelvo null)
 - caso 2: tiene solo un hijo
 - devuelvo ese hijo (lo “subo”)
 - caso 3: tiene dos hijos
 - busco el mínimo del subárbol derecho (el sucesor)
 - reemplazo el valor del nodo por ese sucesor y después elimino ese sucesor del subárbol derecho.
 - Devuelvo el nodo actualizado

Ejercicio 7 - Validación de Rango en ABB

Implementa un método boolean enRango(int desde, int hasta); que indica si el ABB está contenido en el rango indicado (incluyendo los bordes).

Ejercicio 8 - Implementar un Conjunto usando ABB

Crear una clase **ConjuntoABB** que internamente utilice un ABB.

Operaciones básicas:

- void agregar(int valor)
- bool eliminar(int valor)
- bool contiene(int valor)
- int cantidad()
- int elegir()
- ConjuntoABB interseccion(ConjuntoABB conjunto)

Recordar que los conjuntos no permiten duplicados.

Ejercicio 9 - Índice de palabras (Desafío)

Implementar un índice de palabras para un libro, contando cuantas veces aparece cada palabra. Tienen un ejemplo Texto.txt que pueden usar.

Por ejemplo:

"casa árbol casa río árbol árbol"

Resultado:

árbol -> 3
casa -> 2
río -> 1

Utilizar un ABB donde cada nodo almacena:

- Palabra
- cantidadDeApariciones

y mostrar el resultado utilizando recorrido inorden.